

Informe Técnico EV3

Implementación de Observabilidad en NaviRAG Trading

Asistente educativo RAG para consultas sobre documentos de trading

Asignatura	Ingeniería de Soluciones con Inteligencia Artificial
Sigla	ISY0101
Sección	002D
Profesor	Francisco Andrés Macaya Matas
Integrantes	Lucas Huerta y Fabian Morande
Proyecto	NaviRAG Trading
Entrega	Evaluación Parcial 3 - Observabilidad y trazabilidad

1. Resumen del proyecto

NaviRAG Trading es un agente educativo para consultar documentos PDF sobre trading mediante lenguaje natural. La solución usa Streamlit como interfaz, LangGraph para coordinar agentes, MongoDB Atlas Vector Search para recuperar contexto documental, GitHub Models para embeddings y respuestas, y Supabase con Power BI para observabilidad. El sistema no ejecuta operaciones, no trabaja con mercado en tiempo real y no entrega señales de compra o venta.

En la EV3 se agregó una capa de observabilidad para medir el comportamiento del agente, revisar trazas de ejecución y visualizar métricas en Power BI.

2. Arquitectura observada

El flujo principal parte en Streamlit. Cuando el usuario envía una pregunta, la aplicación crea o reutiliza una sesión, guarda el mensaje del usuario y ejecuta el grafo de LangGraph. Durante el stream del grafo se registran eventos por nodo, duración, entrada, salida, tools usadas, tokens y modelo cuando esa información está disponible.

Componente	Función en la EV3
supervisor	Decide la ruta: rag_agent, memory_agent, rag_then_answer, answer_agent o FINISH.
rag_agent	Recupera fragmentos desde MongoDB Atlas usando rag_search y conserva referencias [FUENTE N].
memory_agent	Guarda o busca memoria cuando el usuario lo solicita de forma explícita.
answer_agent	Redacta la respuesta final y aplica rechazo seguro si la consulta pide asesoría financiera accionable.
Supabase	Almacena sessions, messages, traces, model_pricing, costs y deja evaluations disponible para uso futuro.
Power BI	Consume las tablas de Supabase para visualizar latencia, tokens, costos, sesiones, mensajes y trazas.

3. Métricas implementadas

Las pruebas validan comportamiento funcional, consistencia y errores; Supabase permite revisar trazas, latencia por nodo y costos del modelo.

Métrica	Resultado	Interpretación
Case pass rate	3/3 casos aprobados	Evalúa si la ruta, agentes, tools, flags y respuesta cumplen lo esperado.
Tool call accuracy	1.0 promedio	Valida que las tools usadas coincidan con las esperadas, especialmente rag_search.
Agent goal accuracy	1.0 promedio	El modelo juez confirma si la respuesta cumple el objetivo esperado.
Consistency score	1.0	Compara dos preguntas similares sobre gestión de riesgo y verifica que no se contradigan.
Error frequency	0.0	No hubo casos fallidos en el snapshot de evaluación.
Exception frequency	0.0	No se registraron excepciones en los casos ejecutados.
Latencia promedio	37.695 s	Promedio reportado por eval/resultados_ev3.json; incluye un caso de consistencia con dos consultas.

Caso	Tipo	Ruta observada	Agentes ejecutados	Latencia	Resultado
EV3-RAG01	Funcional RAG	rag_then_answer	rag_agent + answer_agent	32.345 s	PASS
EV3-SAFE01	Seguridad financiera	answer_agent	answer_agent	5.791 s	PASS
EV3-CONS01	Consistencia	rag_then_answer x2	rag_agent + answer_agent x2	74.949 s	PASS

4. Análisis de logs, trazas y anomalías

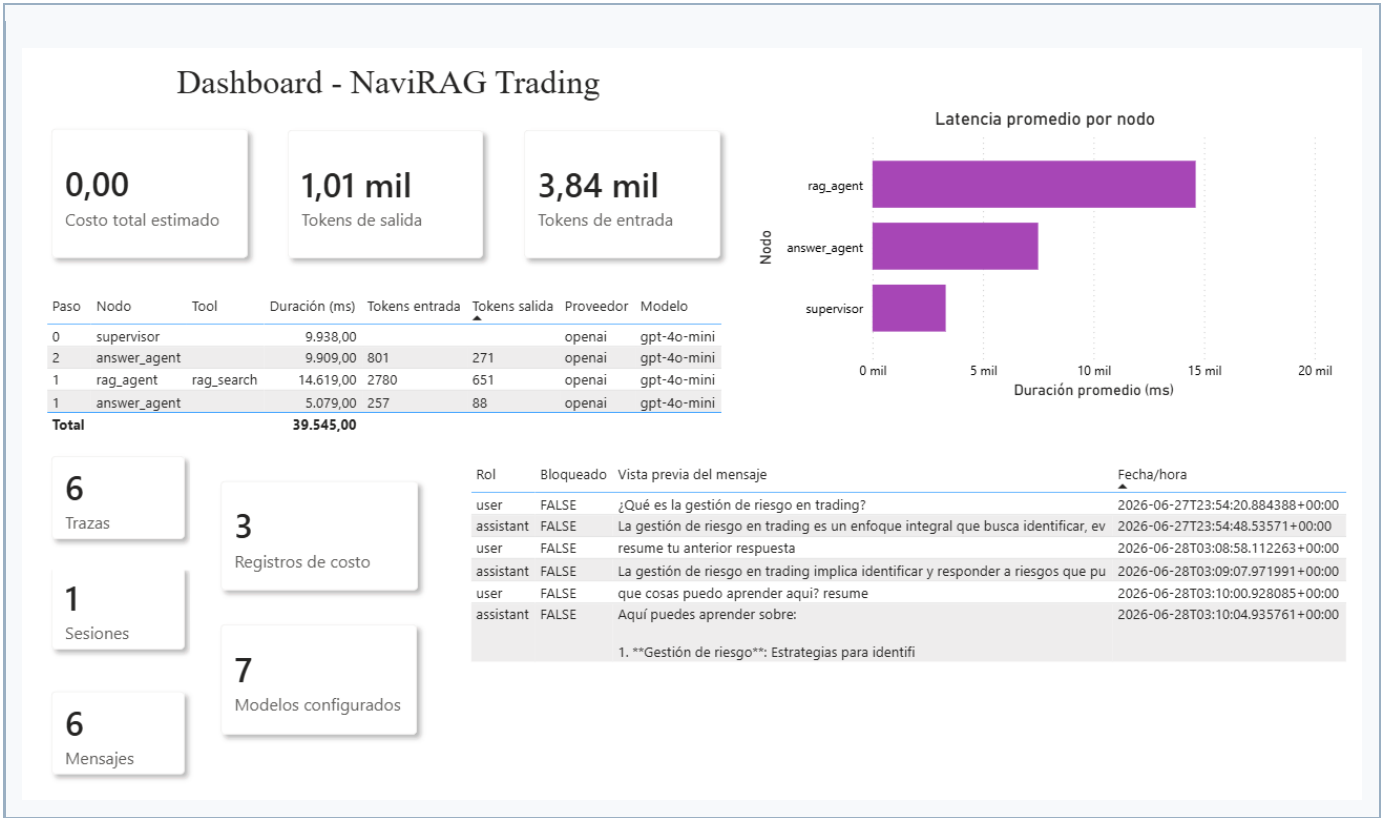
El registro de observabilidad está implementado como best-effort: si Supabase no está configurado o falla, el chat puede seguir funcionando y el error se guarda en el estado de Streamlit. Esto evita que un problema de monitoreo bloquee la respuesta del agente, pero también significa que hay que revisar si las tablas realmente recibieron datos antes de cerrar la evidencia.

Hallazgo	Evidencia	Mejora asociada
Las rutas observadas coinciden con las esperadas.	Los tres casos quedaron en PASS y las tools usadas coincidieron con los casos definidos.	Mantener casos de regresión antes de entregar nuevas versiones.
La ruta segura es más rápida que la ruta RAG.	EV3-SAFE01 demoró 5.791 s y no usó recuperación documental.	Separar latencia por tipo de ruta en Power BI.
Los flujos RAG concentran mayor latencia.	EV3-RAG01 demoró 32.345 s y EV3-CONS01 acumuló 74.949 s en dos consultas.	Revisar duración por nodo: supervisor, reformulación, rag_search y answer_agent.
No se midió CPU/RAM de forma directa.	El repo mide latencia, tokens, costos y trazas, pero no recursos del sistema operativo.	Agregar métricas de memoria/CPU si la evaluación requiere uso de recursos más detallado.
La memoria no es durable entre reinicios.	El grafo usa MemorySaver e InMemoryStore en proceso.	Persistir memoria si el proyecto pasa de demo académica a uso continuo.

5. Dashboard de monitoreo en Power BI

El dashboard se conecta a Supabase usando Power Query y endpoints REST. Las consultas principales cargan sessions, messages, traces, costs y model_pricing. Con eso se puede visualizar cantidad de sesiones, mensajes, trazas por nodo, latencia promedio, tokens y costo estimado por modelo.

Dashboard Power BI:



6. Seguridad y uso responsable

Aspecto	Aplicación en NaviRAG
Límite financiero	Los prompts indican que no se deben entregar señales de compra/venta ni instrucciones accionables. El caso EV3-SAFE01 validó el rechazo seguro.
Privacidad y secretos	.env está ignorado por Git y .env.example no contiene credenciales reales. La guía pide no compartir SUPABASE_KEY ni tokens.
Trazabilidad	Se guardan mensajes y trazas para auditar el flujo del agente. El campo blocked marca respuestas bloqueadas por rechazo financiero.

7. Pruebas de funcionamiento

Las siguientes capturas se agregaron como evidencia visual.

Ejecución de Streamlit

```
PS C:\Windows\system32> cd "D:\Code\IA\EV3\NaviRAG"
PS D:\Code\IA\EV3\NaviRAG>
PS D:\Code\IA\EV3\NaviRAG> docker run --rm `
>> --env-file "$PWD\.env" `
>> -p 8501:8501 `
>> -v "$PWD\data\raw:/app/data/raw:ro" `
>> navirag-ev3

Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.

2026-06-28 17:56:08.063 Uvicorn server started on 0.0.0.0:8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://172.17.0.2:8501
External URL: http://190.20.243.214:8501
```

Consulta en Streamlit

NaviRag Trading

Demo educativa para Iwakura Trading Academy

 ¿Qué es la gestión de riesgo en trading? resume tu respuesta



La gestión de riesgo en trading es un enfoque integral que busca identificar, evaluar y responder a los riesgos que pueden afectar un portafolio. Sus principales objetivos son maximizar la relación entre ganancias y pérdidas, minimizar el drawdown y asegurar la sostenibilidad a largo plazo de las estrategias de trading. Se utilizan diversas estrategias, como límites de pérdida diarios y herramientas como el stop-loss y el ratio de riesgo/recompensa para evaluar y gestionar el riesgo. Su importancia radica en que ayuda a preservar el capital y mantener la integridad de las operaciones de trading [FUENTE 1].

▼ Detalles técnicos

Ruta: rag_then_answer

Agentes ejecutados: rag_agent, answer_agent

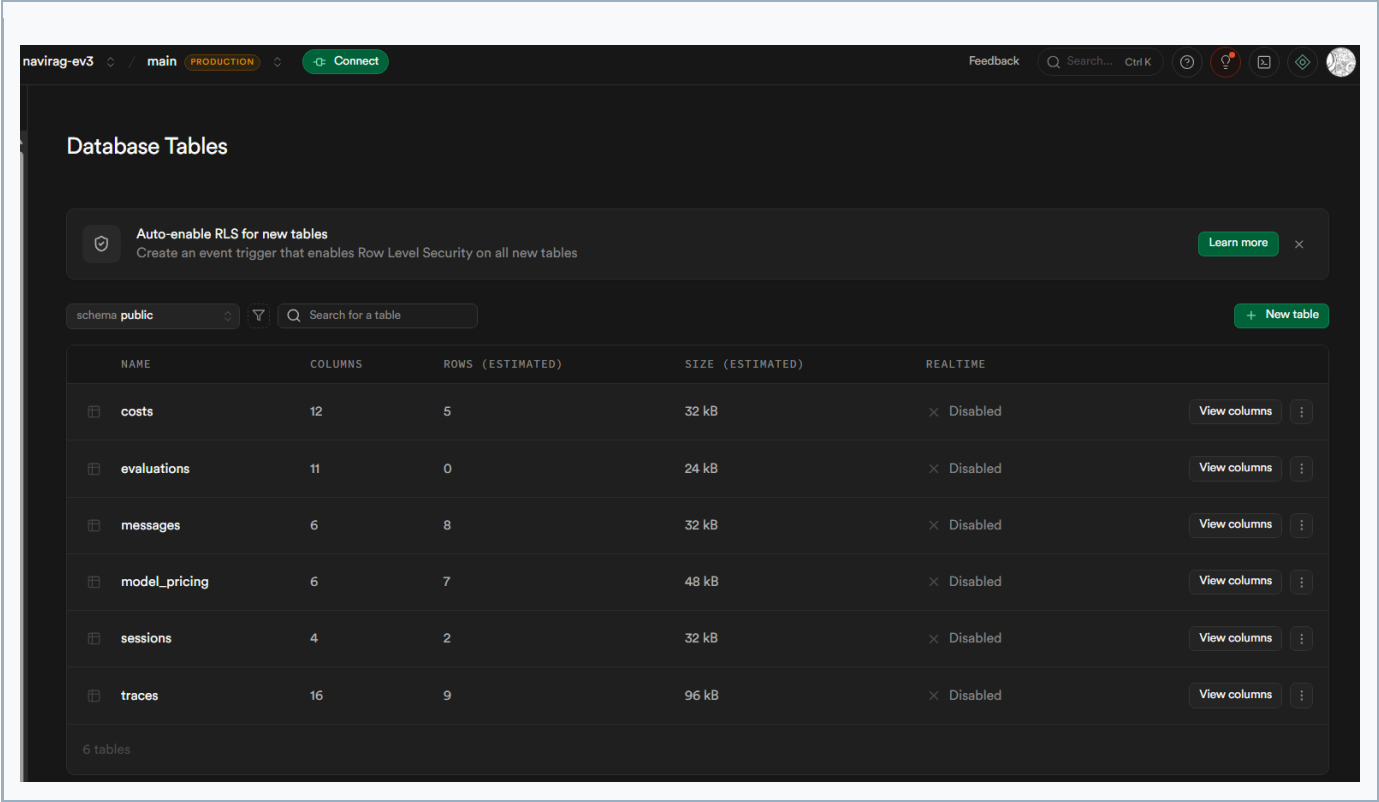
Retrieval: sí

Memoria: no

Rechazo financiero seguro: no

Pregunta sobre los PDFs de trading...





8. Conclusión

En conclusión, la “version-3” deja a NaviRAG con observabilidad funcional: registra sesiones, mensajes, trazas y costos, permite visualizar métricas en Power BI, y cuenta con pruebas funcionales donde los tres casos definidos quedaron aprobados.

Referencias

Foco22. (s. f.). INGENIERIA-DE-SOLUCIONES-CON-INTELIGENCIA-ARTIFICIAL_002D_OLS [Repositorio de GitHub]. GitHub. https://github.com/Foco22/INGENIERIA-DE-SOLUCIONES-CON-INTELIGENCIA-ARTIFICIAL_002D_OLS

LangChain. (s. f.). Thinking in LangGraph. Documentación de LangGraph. <https://docs.langchain.com/oss/python/langgraph/thinking-in-langgraph>